Physics-Informed Neural Networks for Track Extrapolation: A Literature Review and Comparison with V1 Results

George William Scriven

Gravitational Waves and Fundamental Physics (GWFP), Maastricht University, The Netherlands

UHasselt, Hasselt University, Belgium

LHCb, CERN, Geneva, Switzerland

George.Scriven@uhasselt.be

George.William.Scriven@cern.ch

March 2026

Abstract

This document reviews four papers from the physics-informed neural network (PINN) literature and places them in the context of our V1 neural track extrapolation study for the LHCb experiment. The V1 study trained 75+ neural network models — including MLPs, PINNs, and a novel RK-PINN architecture — to replace the Runge–Kutta 4 (RK4) track extrapolator used in LHCb's reconstruction pipeline. While pure MLPs achieved a 2.9× speedup over C++ with sub-100 μm accuracy, physics-informed approaches failed catastrophically due to an architectural flaw. We examine how techniques from the recent PINN literature — finite-difference-based physics losses, data-free training, temporal domain decomposition, and RK-structured networks — address the specific challenges encountered in V1 and could inform future iterations.

Contents

1 Introduction **3**

2 Summary of V1 Approach and Results **3**

2.1 Architectures 3

2.2 Training 4

2.3 Key Results 4

3 Literature Review **4**

3.1 Paper 1: FD-PINN — Finite Difference Physics Loss 4

3.2 Paper 2: Data-Free PINNs for Time-Domain Simulation 5

3.3 Paper 3: Long-Time Integration with Physics-Informed DeepONets 5
3.4 Paper 4: RK-PINN for Parameter Estimation 6

4 Detailed Comparison with V1 **6**
4.1 Paper 1 (FD-PINN) vs. V1 6
4.2 Paper 2 (Data-Free PINNs) vs. V1 7
4.3 Paper 3 (DeepONet Domain Decomposition) vs. V1 8
4.4 Paper 4 (RK-PINN) vs. V1 9

5 Synthesis: A Roadmap for V2+ **10**

6 Conclusions **11**

1 Introduction

Track extrapolation in the LHCb detector requires solving the Lorentz force ordinary differential equations (ODEs) that govern charged particle motion through a non-uniform magnetic field:

$$\frac{dx}{dz} = t_x, \qquad \frac{dy}{dz} = t_y, \tag{1}$$

$$\frac{dt_x}{dz} = \kappa N \left[t_x t_y B_x - (1 + t_x^2) B_y + t_y B_z \right], \tag{2}$$

$$\frac{dt_y}{dz} = \kappa N \left[(1 + t_y^2) B_x - t_x t_y B_y - t_x B_z \right], \tag{3}$$

where $\kappa = (q/p) \cdot c_{\text{light}}$, $N = \sqrt{1 + t_x^2 + t_y^2}$, and $\boldsymbol{B}(x, y, z)$ is the LHCb dipole field. In production, this system is solved numerically using RK4 integration with trilinear interpolation of a tabulated field map (`twodip.rtf`), at a cost of approximately $2.50\,\mu$s per track.

The V1 study explored whether neural networks can learn this mapping

$$f_\theta : (x_0, y_0, t_{x,0}, t_{y,0}, q/p, \Delta z) \longrightarrow (x_f, y_f, t_{x,f}, t_{y,f}) \tag{4}$$

with sufficient accuracy and speed to replace the C++ extrapolator in the LHCb Upgrade II trigger. Three architecture families were investigated: standard MLPs, physics-informed neural networks (PINNs), and a novel Runge–Kutta-structured PINN (RK-PINN). A total of 75 model configurations were trained on 50 million track samples generated by the reference RK4 integrator.

2 Summary of V1 Approach and Results

2.1 Architectures

MLP. Standard feedforward networks with SiLU activation, trained with pure data loss (MSE). Seven size variants were explored, from *tiny* ([64, 32], 5k params) to *wide* ([512, 512, 256, 128], 431k params).

PINN. The same backbone augmented with a physics-informed loss:

$$\mathcal{L}_{\text{PINN}} = \mathcal{L}_{\text{data}} + \lambda_{\text{IC}}\, \mathcal{L}_{\text{IC}} + \lambda_{\text{PDE}}\, \mathcal{L}_{\text{PDE}}, \tag{5}$$

where $\mathcal{L}_{\text{PDE}}$ is the Lorentz force residual computed via automatic differentiation at $N_c = 10$ collocation points along the trajectory, and $\mathcal{L}_{\text{IC}} = \| f_\theta(\boldsymbol{x}_0, z{=}0) - \boldsymbol{x}_0 \|^2$ enforces initial condition satisfaction. A simplified Gaussian magnetic field model ($B_y(z) = B_0 \exp[-(z - z_c)^2/2\sigma_z^2]$, $B_0 = -1.02\,\text{T}$) was used for differentiability.

RK-PINN. A novel multi-stage architecture inspired by RK4 numerical integration. A shared trunk network extracts features, and four stage-specific heads predict state updates at $z_{\text{frac}} \in \{0.25, 0.5, 0.75, 1.0\}$. Stage outputs are combined via learnable weights initialised to the classical RK4 coefficients [1/6, 1/3, 1/3, 1/6] and passed through a softmax:

$$\hat{\boldsymbol{s}}_{\text{out}} = \boldsymbol{s}_{\text{in}} + \sum_{k=1}^{4} w_k(\boldsymbol{\alpha})\, \Delta \boldsymbol{s}_k, \qquad w_k = \frac{\exp(\alpha_k)}{\sum_j \exp(\alpha_j)}. \tag{6}$$

2.2 Training

All models were trained on 50 M samples generated with the reference RK4 integrator and the full LHCb field map. Training used AdamW (lr = 10^{-3}, $\lambda_{\text{wd}} = 10^{-4}$), cosine annealing, batch size 8192, and up to 200 epochs with early stopping. PINN-specific stability features included gradient clipping (threshold 1.0), physics loss warmup over the first 10 epochs, and NaN/Inf detection.

2.3 Key Results

Table 1: V1 benchmark results: selected models vs. C++ extrapolators.

Model	**Type**	**Params**	**Time (μs)**	**Error (μm)**
mlp_tiny	MLP	4.9k	1.12	53.5
mlp_small	MLP	17.9k	1.22	53.5
mlp_medium	MLP	101k	2.17	37.2
pinn_medium	PINN	101k	2.46	$> 10^5$
rkpinn_medium	RK-PINN	202k	5.59	$> 10^5$
Kisel	C++	—	1.50	39 800
Herab	C++	—	1.95	759.6
BogackiShampine3	C++	—	2.40	101.4
Reference RK4	C++	—	2.50	0 (ref.)

The principal findings were:

1. **MLPs are optimal.** Small MLPs achieved 2.9× speedup over C++ RK4 with sub-100 μm accuracy.

2. **Physics-informed approaches failed.** Both PINNs and RK-PINNs produced >500 mm errors — approximately $10^4\times$ worse than MLPs.

3. **Root cause: IC violation.** The PINN `forward()` method hardcoded $z_{\text{frac}} = 1.0$, so the network never learned intermediate trajectory points. Physics loss at collocation points evaluated an untrained regime, and the network learned a constant output independent of z.

4. **Proposed fix (PINN_v3).** A residual architecture $\boldsymbol{y}(z) = \boldsymbol{y}_0 + z_{\text{frac}} \cdot \text{MLP}(\boldsymbol{y}_0, q/p, \Delta z)$ guarantees IC satisfaction by construction. Not yet trained at the time of V1 publication.

3 Literature Review

3.1 Paper 1: FD-PINN — Finite Difference Physics Loss

"Physics-informed deep learning based on the finite difference method for efficient and accurate numerical solution of partial differential equations."

Summary. This paper proposes replacing automatic differentiation (AD) in the PINN physics loss with classical finite difference (FD) stencils. Standard PINNs compute the

PDE residual by differentiating the network output with respect to its inputs via backpropagation. FD-PINNs instead evaluate the network at neighbouring points and approximate derivatives numerically:

$$\left.\frac{\partial \hat{u}}{\partial x}\right|_{x_i} \approx \frac{\hat{u}(x_i+\delta)-\hat{u}(x_i-\delta)}{2\delta}. \tag{7}$$

The authors demonstrate that FD-PINNs achieve comparable or better accuracy on benchmark PDEs (Burgers' equation, convection–diffusion, Navier–Stokes) while being significantly faster to train, since FD avoids the memory overhead of storing the computational graph for second-order backpropagation.

Key contributions:

- Eliminates the need for automatic differentiation in the physics loss, reducing memory consumption and training time.
- The network architecture is decoupled from the physics loss computation — any black-box function evaluator can provide the derivative approximation.
- Naturally accommodates non-differentiable components (e.g. tabulated data, lookup tables) in the governing equations.

3.2 Paper 2: Data-Free PINNs for Time-Domain Simulation

"Learning without Data: Physics-Informed Neural Networks for Fast Time-Domain Simulation." Misyris, Stiasny, and Chatzivasileiadis, IEEE SmartGridComm (2021).

Summary. This paper demonstrates that PINNs can solve systems of differential algebraic equations (DAEs) for power grid simulation *entirely without training data*. The network is trained by minimising only the ODE/PDE residual and initial/boundary conditions:

$$\mathcal{L} = \lambda_{\text{PDE}}\,\mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\,\mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\,\mathcal{L}_{\text{BC}}. \tag{8}$$

Once trained, the network provides orders-of-magnitude speedup over traditional numerical solvers and can generalise to parametric families of initial conditions.

Key contributions:

- Eliminates the need for a numerical solver to generate training data; the physics alone supervises learning.
- Demonstrates that a single trained network can solve families of related problems without retraining.
- Inference is orders of magnitude faster than iterative numerical solvers.

3.3 Paper 3: Long-Time Integration with Physics-Informed DeepONets

"Long-time integration of parametric evolution equations with physics-informed DeepONets."

Summary. Standard PINNs struggle with long-time integration because the optimisation landscape becomes increasingly complex as the time horizon grows. This paper addresses

the problem using **DeepONets** (Deep Operator Networks) — a neural operator architecture that learns mappings between function spaces — combined with **temporal domain decomposition**. The time domain is split into short windows; the operator network predicts each window's solution, and outputs are chained sequentially. This avoids the spectral bias that causes standard PINNs to fail on long-range dynamics.

Key contributions:

- **Operator learning:** The DeepONet learns the solution operator $\mathcal{G} : u_0 \mapsto u(t)$, not a single solution instance. One trained network handles all initial conditions.
- **Temporal decomposition:** Splitting long integrations into short, independently solvable windows controls error accumulation.
- **Parametric generalisation:** The trained operator generalises across different initial conditions and equation parameters without retraining.

3.4 Paper 4: RK-PINN for Parameter Estimation

"Parameter estimation and modeling of nonlinear dynamical systems based on Runge–Kutta physics-informed neural network."

Summary. This paper embeds the Runge–Kutta numerical integration structure directly into a neural network for both forward simulation and inverse parameter estimation. Each network "stage" predicts a derivative evaluation at an intermediate point, and stages are combined using learnable weights initialised to the classical RK4 Butcher tableau coefficients. The architecture handles both forward problems (given parameters, predict trajectory) and inverse problems (given trajectory, estimate unknown system parameters) within a unified framework.

Key contributions:

- The RK structure acts as a strong inductive bias, ensuring the network respects the temporal evolution structure of the ODE.
- Learnable combination weights allow the network to adapt beyond classical RK4 while retaining its convergence properties as initialisation.
- Dual forward/inverse capability enables simultaneous trajectory prediction and parameter estimation from sparse, noisy observations.

4 Detailed Comparison with V1

4.1 Paper 1 (FD-PINN) vs. V1

Key similarity. Both approaches embed PDE residuals into the training loss to regularise the network towards physically consistent solutions.

Key difference. V1 requires a differentiable field model for AD, creating a *fundamental mismatch*: training data is generated with the full interpolated field map, but the physics loss enforces a simplified Gaussian approximation. FD-PINNs eliminate this mismatch entirely because finite differences only require function evaluations at discrete points — the tabulated field map can be used directly.

Table 2: FD-PINN vs. V1 PINN: physics loss computation.

Aspect	**V1 PINN**	**FD-PINN (Paper 1)**
Derivative computation	Automatic differentiation (`torch.autograd`)	Finite difference stencils
Differentiable field required?	Yes — forced use of Gaussian approximation	No — any field evaluator works
Memory overhead	High (stores full computational graph)	Low (only forward passes)
Training speed	$\sim$3$\times$ slower than MLP	Comparable to MLP
Field model	Gaussian: $B_y = B_0 e^{-(z-z_c)^2/2\sigma_z^2}$	Can use tabulated `twodip.rtf` directly

Implication for V2+. Adopting FD-based physics loss would:

1. Remove the field model mismatch (the dominant source of physics loss ineffectiveness in V1).
2. Reduce PINN training time from $\sim$3$\times$ to $\sim$1.2$\times$ that of MLPs.
3. Enable physics-informed training with the production field map, eliminating a major barrier to deployment.

4.2 Paper 2 (Data-Free PINNs) vs. V1

Table 3: Data-free PINN vs. V1: training paradigm.

Aspect	**V1**	**Paper 2 (Data-Free)**
Training data	50M samples from C++ RK4	Zero — physics loss only
Data generation cost	High (C++ infrastructure)	None
Loss function	$\mathcal{L}_{\text{data}} + \lambda\,\mathcal{L}_{\text{PDE}}$	$\mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{IC}}$ only
ODE system	Lorentz force (4 coupled ODEs)	Power grid DAEs
Input dimensionality	6D (high)	Typically 1–3D (low)
Parameter variation	$q/p \in [0.01, 1.0]\,\text{GeV}^{-1}$	Fixed or low-dimensional

Key similarity. Both approaches seek to train neural networks as fast surrogates for numerical ODE/DAE solvers, using the governing equations as supervision signals.

Key difference. V1 operates in a *data-rich* regime (50M samples) where the data loss alone is sufficient for accurate learning, making the physics loss redundant. Paper 2 operates in a *data-free* regime where the physics is the only supervision. The LHCb problem also has much higher input dimensionality (6D vs. 1–3D) and a wider parameter range.

Implication for V2+. A pure data-free approach is unlikely to work for the full LHCb problem due to the high dimensionality and complex, spatially varying field. However, a *hybrid* strategy is attractive:

- Use a small dataset (∼1–5M samples) for coarse fitting, supplemented by physics loss for fine-grained regularisation.
- This would decouple the ML pipeline from the C++ data generation infrastructure, enabling faster iteration when the field map or detector geometry changes.
- The PINN_v3 residual formulation is essential for this — Paper 2 implicitly assumes a network architecture that satisfies initial conditions, which V1's original PINN did not.

4.3 Paper 3 (DeepONet Domain Decomposition) vs. V1

Table 4: DeepONet temporal decomposition vs. V1: handling variable integration length.

Aspect	**V1**	**Paper 3 (DeepONet)**
Integration range	Fixed $\Delta z = 8000\,\mathrm{mm}$	Arbitrary (decomposed)
Variable step handling	Single network for all Δz (V3 attempt: poor)	Short-window chaining
Architecture	MLP / RK-PINN	DeepONet (branch + trunk)
Error accumulation	Single-step (no chaining)	Controlled via short windows
Generalisation	Poor beyond training Δz	By construction (operator)

Key similarity. Both address the fundamental challenge of learning ODE solutions over extended domains. V1's failure to generalise from fixed $\Delta z = 8000\,\mathrm{mm}$ to variable step sizes is precisely the "long-time integration" problem that Paper 3 solves.

Key difference. V1 attempted to learn a monolithic mapping for all step sizes simultaneously (V3), resulting in ∼1 mm errors. Paper 3 avoids this by decomposing the domain into short windows and chaining predictions. This mirrors how the actual RK4 integrator works internally (many small steps within a single extrapolation call).

Implication for V2+. Domain decomposition is the most directly applicable technique from the literature:

1. Train a high-accuracy short-step model (e.g. $\Delta z = 500$–$1000\,\mathrm{mm}$).
2. Chain N applications for longer extrapolations ($N \times \Delta z_{\mathrm{short}}$).
3. Error accumulation is bounded by N times the single-step error.
4. This is *naturally compatible* with the Kalman filter, which calls the extrapolator sequentially between detector planes with known z positions.

The risk is that chaining introduces N sequential inference steps, potentially negating the speed advantage. However, if each short step costs $\sim$0.5 μs and only 2–4 steps are needed per inter-plane gap, the total ($\sim$1–2 μs) would still match or beat the C++ RK4 baseline (2.50 μs).

The operator learning framing (DeepONet) also suggests a conceptual shift: rather than learning a single solution, learn the *solution operator* that maps any initial state to its evolved state. This naturally handles the parametric variation in $(q/p, \boldsymbol{x}_0)$ that V1 addressed through brute-force data coverage.

4.4 Paper 4 (RK-PINN) vs. V1

Table 5: Literature RK-PINN vs. V1 RK-PINN: architecture and capabilities.

Aspect	**V1 RK-PINN**	**Paper 4 RK-PINN**
Stage structure	4 heads at $z = \{0.25, 0.5, 0.75, 1\}\Delta z$	Multi-stage RK derivative predictions
Weight initialisation	Softmax of learnable $\alpha_k \to$ RK4	Butcher tableau coefficients
Physics loss	Soft t_y conservation only	Full PDE residual at each stage
IC satisfaction	**Not guaranteed** (root cause of failure)	Assumed (residual formulation)
Inverse problems	Not explored	Parameter estimation from data
Forward accuracy	2042 mm (failed)	Accurate on test problems

Key similarity. V1's RK-PINN is a direct implementation of the ideas in Paper 4. Both use multi-stage architectures with learnable combination weights initialised to RK4 coefficients. Both aim to encode numerical integration structure as an inductive bias.

Key difference. The V1 implementation suffered from the same IC violation bug as the standard PINN: the network could produce arbitrary outputs at $z = 0$, violating the fundamental requirement $\hat{\boldsymbol{s}}(z{=}0) = \boldsymbol{s}_0$. Paper 4's architecture implicitly assumes IC satisfaction (through a residual form or explicit enforcement), which V1 did not implement. Additionally, V1 used only a weak t_y conservation constraint rather than the full Lorentz force PDE residual.

Implication for V2+.

1. The RK-PINN architecture deserves re-evaluation with the PINN_v3 residual fix. The multi-stage structure could provide better generalisation for variable step sizes than a flat MLP, since it naturally decomposes the integration.

2. The full PDE residual should be enforced at each RK stage (as in Paper 4), not just a soft t_y constraint. Combined with the FD approach from Paper 1, this becomes computationally feasible with the production field map.

3. The **inverse problem capability** is particularly valuable for LHCb. Given observed track hits across detector planes, an RK-PINN could simultaneously:

   - Estimate q/p (momentum measurement — currently done by the Kalman filter).
   - Refine the magnetic field map from track residuals (calibration).
   - Estimate detector alignment parameters.

   This would unify the extrapolation and fitting steps that are currently separate in the reconstruction chain.

5 Synthesis: A Roadmap for V2+

The four papers collectively address every major failure mode encountered in V1:

Table 6: V1 failure modes and literature solutions.

V1 Failure Mode	**Paper**	**Proposed Solution**
IC violation (PINN ignores z)	Papers 2, 4	Residual architecture (PINN_v3)
Field model mismatch (Gaussian $\neq$ real)	Paper 1	FD-based physics loss
Fixed Δz (no generalisation)	Paper 3	Temporal domain decomposition
Weak physics constraints (t_y only)	Paper 4	Full PDE residual at each RK stage
No inverse capability	Paper 4	Dual forward/inverse RK-PINN
High data requirement (50M samples)	Paper 2	Hybrid low-data + physics loss

Priority ranking.

1. **[High] Fix and retrain PINN_v3 + RK-PINN_v3** with guaranteed IC satisfaction. This is the prerequisite for all other improvements and validates whether the physics-informed approach works at all for this problem.

2. **[High] Adopt FD-based physics loss** (Paper 1). Removes the field model mismatch and enables training with the production field map. Low implementation effort.

3. **[Medium] Implement short-step chaining** (Paper 3). Solves the variable-Δz generalisation problem. Naturally compatible with the Kalman filter's sequential extrapolation structure.

4. **[Exploratory] Inverse problem / Kalman filter unification** (Paper 4). High risk but transformative — could replace both the extrapolator and parts of the Kalman filter with a single differentiable model.

6 Conclusions

The V1 study demonstrated that neural networks can match or exceed the speed of C++ track extrapolators while maintaining sub-100 μm accuracy, but only when using pure data-driven MLPs. The failure of physics-informed approaches was not fundamental but architectural: an IC violation caused PINNs and RK-PINNs to produce meaningless predictions.

The four papers reviewed here provide a clear path beyond this failure. The finite difference approach (Paper 1) eliminates the field model mismatch that forced V1 to use a simplified Gaussian field. Data-free training (Paper 2) demonstrates that physics-only supervision can work for ODE systems, given a correct architecture. Temporal domain decomposition (Paper 3) directly addresses V1's inability to generalise across step sizes. And the literature RK-PINN (Paper 4) validates the architectural concept while highlighting what V1's implementation missed: guaranteed IC satisfaction and full PDE enforcement at every stage.

The most promising near-term path combines the PINN_v3 residual fix with FD-based physics loss and short-step chaining. This would yield a physics-regularised neural extrapolator that is accurate, fast, generalisable across step sizes, and trainable directly with the production field map — addressing every limitation of V1 simultaneously.

References

[1] "Physics-informed deep learning based on the finite difference method for efficient and accurate numerical solution of partial differential equations."

[2] G.S. Misyris, A. Stiasny, and S. Chatzivasileiadis, "Learning without Data: Physics-Informed Neural Networks for Fast Time-Domain Simulation," *IEEE SmartGridComm* (2021). doi:10.1109/SmartGridComm51999.2021.9631995

[3] "Long-time integration of parametric evolution equations with physics-informed DeepONets."

[4] "Parameter estimation and modeling of nonlinear dynamical systems based on Runge–Kutta physics-informed neural network."

[5] M. Raissi, P. Perdikaris, and G.E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.* **378**, 686 (2019). doi:10.1016/j.jcp.2018.10.045

[6] G.E. Karniadakis et al., "Physics-informed machine learning," *Nature Rev. Phys.* **3**, 422 (2021). doi:10.1038/s42254-021-00314-5

[7] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations," *NeurIPS* (2018). arXiv:1806.07366